

IMT Atlantique

Technopôle de Brest-Iroise - CS 83818

29238 Brest Cedex 3

Téléphone : +33 (0)2 29 00 13 04

Télécopie : +33 (0)2 29 00 10 12

URL : www.imt-atlantique.fr



Document Développeur

Etienne BERTIN

Christ-Roi SEYA

Kani FOMBA

Aloïs VINCENT

Xiangyu MAO

Emmanuel Rachid Junior KOUASSI

Date d'édition : 22 janvier 2026

**IMT Atlantique**

Bretagne-Pays de la Loire

École Mines-Télécom

1. Introduction

Le projet TenukiGo s'inscrit dans une démarche d'innovation destinée à moderniser la pratique et la diffusion du jeu de Go au sein du club Tenuki-Brest. L'objectif global est de concevoir un objet connecté autonome capable de capturer une partie de Go en continu via une caméra, de l'enregistrer localement sur une carte SD, de permettre la consultation de ces vidéos via un serveur embarqué, de diffuser en direct l'évolution de la partie sur des plateformes externes. Enfin, l'objet doit être capable d'analyser l'état du plateau afin de générer un fichier SGF automatiquement.

L'intérêt de ce projet est double. D'une part, il répond à un besoin pratique du club : documenter les parties jouées, faciliter leur analyse à posteriori, et améliorer l'expérience des joueurs lors de tournois ou événements. D'autre part, il explore une problématique technique complexe mêlant informatique embarquée, traitement vidéo, configuration réseau dual-mode, et interaction matérielle (caméra, LEDs, boutons, alimentation autonome).

La finalité n'est donc pas uniquement la construction d'un prototype fonctionnel, mais également la structuration d'une architecture technique répliquable et compréhensible pour des développeurs futurs. Le présent document développeur vise précisément à détailler cette architecture, à expliciter les choix techniques réalisés et à fournir une base solide pour la maintenance et l'évolution du système.

2. Matériel et environnement

Le système repose sur une *Raspberry Pi 3B+*, choisie pour son bon rapport puissance/consommation, sa compatibilité avec les modules caméra, ainsi que la richesse de son écosystème logiciel (bibliothèques vidéo, gestion réseau, services embarqués). La Raspberry constitue l'unité centrale du dispositif : elle exécute les scripts de capture vidéo, gère le stockage local et pilote les modes réseau.

La caméra utilisée est un module compatible Raspberry *PiCam*, capable de produire un flux vidéo stable en H264. Les vidéos générées sont stockées sur une carte SD de 16 Go (qui peut aisément être remplacée par une carte SD de 32 à 64 Go, selon le besoin).

Le système repose sur *Raspberry Pi OS Lite*, afin de limiter la consommation de ressources et d'assurer une exécution fluide des scripts. Plusieurs services du système d'exploitation jouent un rôle essentiel, notamment :

- *libcamera* / *ffmpeg* pour la capture vidéo,
- *hostapd* pour l'émission d'un réseau Wi-Fi local en mode point d'accès,
- *dnsmasq* pour la gestion DHCP du réseau local,
- *wpa_supplicant* pour la connexion à un réseau Wi-Fi externe,
- un serveur dédié pour la consultation des vidéos via un navigateur.

Cette configuration hybride permet à l'objet connecté de fonctionner dans deux scénarios distincts :

- un environnement avec réseau disponible (salle de tournoi équipée),
- un environnement isolé où le système doit émettre son propre réseau (*AP mode*) afin de permettre la récupération des vidéos (bar ou ailleurs).

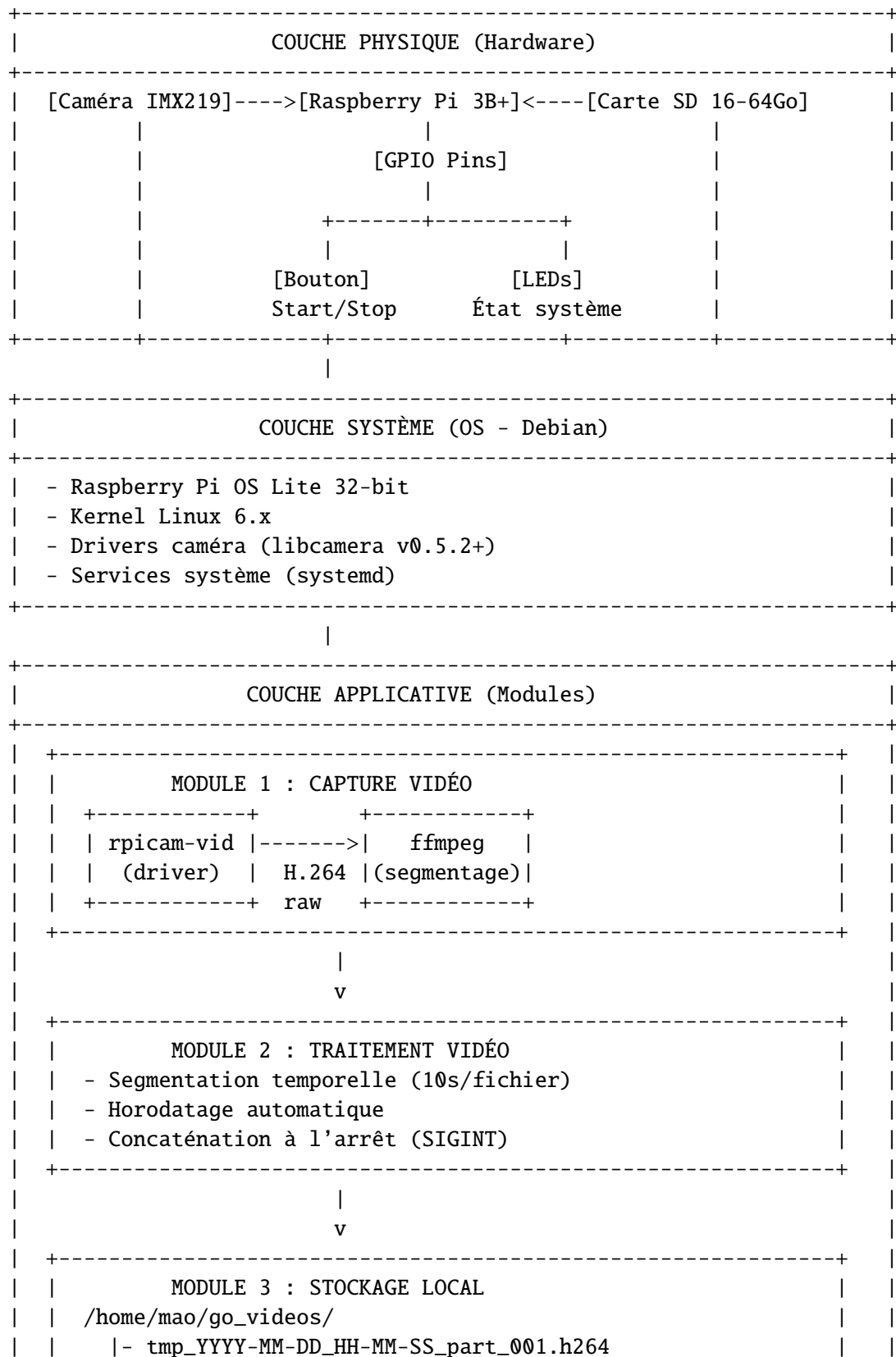
Le matériel est une des grandes limitations de ce projet. Les principaux problèmes étant la puissance de calcul de la Raspberry 3B+ qui est largement insuffisante pour faire tourner localement des algorithmes de computer vision. De plus, les Raspberry n'étaient pas très stables, ce qui nous a fait perdre beaucoup de temps pendant le développement. Enfin, la caméra n'avait pas un angle de vue assez grand, ce qui a ultimement limité le champ de vision de l'objet une fois monté.

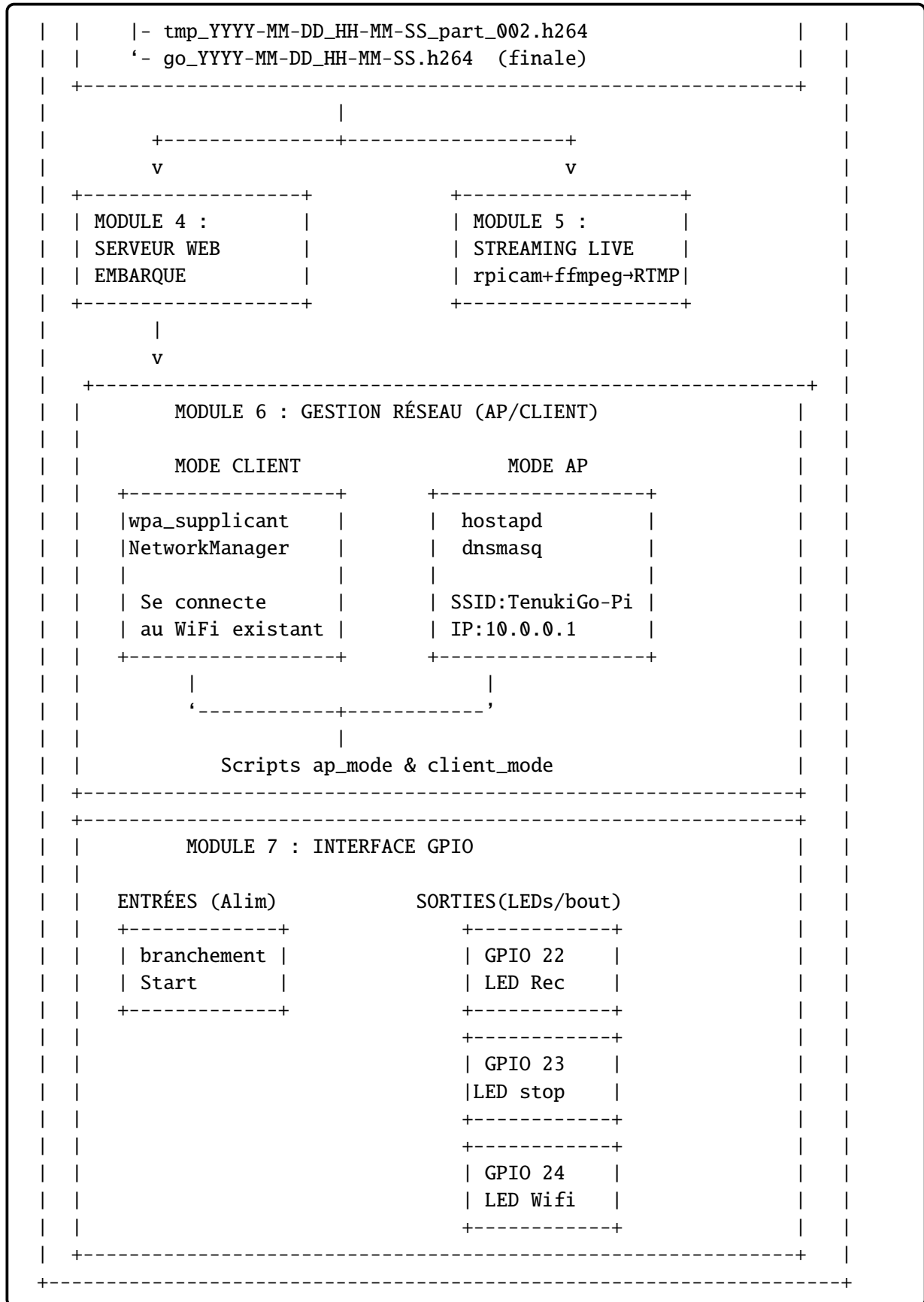
3. Architecture du système

L'architecture globale du système repose sur une organisation modulaire permettant de séparer clairement les responsabilités et de faciliter le développement indépendant des différents sous-systèmes. La conception suit une logique verticale : chaque couche remplit une fonction précise et expose des résultats aux couches supérieures.

3.1. Schéma technique de l'architecture globale

Schéma technique de l'architecture globale





3.2. Description détaillée des modules

Le système est organisé en sept modules principaux :

Module de capture vidéo et traitement vidéo

Ce module regroupe l'ensemble du pipeline vidéo, depuis la capture brute jusqu'à la segmentation des fichiers. La capture est assurée par l'outil `rpicam-vid`, qui fournit une interface bas niveau avec le driver `libcamera` pour la caméra IMX219. Le système enregistre un flux en résolution 1280×720 pixels à 25 images par seconde, encodé en H.264 sans transformation supplémentaire. L'option `-inline` permet l'insertion systématique des en-têtes SPS/PPS dans chaque I-frame, garantissant la décodabilité de chaque segment. Le flux est ensuite redirigé vers la sortie standard (`-o -`) afin d'être traité en temps réel par `ffmpeg`.

L'outil `ffmpeg` lit le flux en temps réel grâce à l'option `-re`, ce qui limite les risques de saturation du processeur. Il effectue ensuite une copie directe du flux vidéo à l'aide du paramètre `-c copy`, afin d'éviter tout ré-encodage. Le découpage est réalisé avec le mode `segment` (`-f segment -segment_time 10`), qui produit des fichiers de durée constante tout en réinitialisant les horodatages pour assurer la cohérence temporelle entre les segments. Un court algorithme de finalisation, déclenché lors d'un signal d'interruption, concatène l'ensemble des segments (`cat tmp_*.h264`) avant de supprimer les fichiers temporaires.

L'ensemble du module est actuellement fonctionnel, mais plusieurs limitations persistent. Du côté de l'acquisition, des timeouts intermittents peuvent survenir si le câble ruban de la caméra n'est pas fermement connecté, et aucun aperçu vidéo ne peut être affiché via SSH, ce qui impose l'utilisation d'un écran HDMI. Enfin, aucune gestion d'erreur avancée n'a encore été intégrée en cas de saturation de la carte SD.

Module de stockage local

La persistance des enregistrements est assurée sur la carte SD en utilisant une nomenclature strictement horodatée, ce qui garantit l'unicité des fichiers et facilite leur tri chronologique. Les vidéos sont stockées dans le répertoire `/home/mao/go_videos/`, où chaque enregistrement produit d'abord une série de segments temporaires du type `tmp_YYYY-MM-DD_HH-MM-SS_part_XXX.h264`. Une fois l'acquisition terminée, ces fragments sont concaténés pour former un fichier final unique `go_YYYY-MM-DD_HH-MM-SS.h264`, puis immédiatement supprimés afin de libérer l'espace disque et de ne conserver que la version définitive de l'enregistrement. Cette convention de nommage, fondée sur un horodatage à la seconde, assure une organisation claire et une récupération aisée des données.

Le système est opérationnel, mais présente encore quelques limites : absence de monitoring de l'espace disponible, absence d'alerte en cas de saturation de la carte, et risque de corruption en cas d'arrêt brutal, du fait de l'absence de mécanismes de journalisation.

Module réseau (AP ↔ Client)

Le module réseau gère la bascule entre deux modes de fonctionnement en fonction du contexte d'utilisation : le mode Client par défaut et le mode Access Point (AP) pour un usage autonome.

Mode Client (par défaut) : la Raspberry Pi se connecte à un réseau WiFi existant (box, routeur du club) grâce aux services `NetworkManager` et `wpa_supplicant`. Dans ce mode, l'appareil bénéficie d'un accès Internet, ce qui permet notamment le streaming live, et reçoit une adresse IP attribuée par le serveur DHCP du réseau local.

Mode Access Point (AP) : la Raspberry Pi crée un point d'accès WiFi autonome avec SSID `WiFi-Tenuki` et mot de passe `Go123456`. L'IP de l'appareil est fixe (`10.0.0.1/24`), et les services `hostapd` et `dnsmasq` gèrent respectivement l'émission du réseau et l'attribution des adresses IP sur la plage `10.0.0.2` à `10.0.0.20`.

La bascule entre les modes est assurée par le script `net_mode.sh` :

```
sudo ~/scripts/net_mode_ap.sh      # Passe en mode AP
sudo ~/scripts/net_mode_client.sh  # Retour mode client
```

En mode AP, la séquence de bascule comprend l'arrêt de `NetworkManager` et `wpa_supplicant`, le flush de la configuration IP de `wlan0`, l'attribution de l'IP statique 10.0.0.1/24, l'activation de l'interface, puis le démarrage de `hostapd`, `dnsmasq` et du serveur HTTP associé au module de diffusion vidéo.

État actuel : Fonctionnel et stable.

Problèmes identifiés : des instabilités ont été observées lors de la connexion de certains terminaux, en particulier les smartphones iOS, pouvant entraîner un dysfonctionnement du service réseau.

pistes pour résoudre :

Module serveur Web embarqué

Streaming live

Le module de streaming live permet la diffusion en temps réel du flux vidéo vers des plateformes comme OGS ou le serveur du groupe 20. La capture vidéo est réalisée via `rpicas-vid` en 720p à 30 images par seconde, et le flux est immédiatement traité par `ffmpeg`. Pour satisfaire les exigences des plateformes de streaming, un flux audio silencieux est généré avec `anullsrc`, le bitstream est converti en format Annex-B avec `h264_mp4toannexb`, puis le tout est encapsulé en FLV et envoyé vers l'endpoint RTMP correspondant.

Le pipeline technique complet s'écrit ainsi :

```
rpicas-vid --width 1280 --height 720 --framerate 30 -o - --inline -n | \  
ffmpeg -i - -f lavfi -i anullsrc \  
-c:v copy -b:v 3000k -bsf:v h264_mp4toannexb \  
-c:a aac -b:a 128k -ar 44100 \  
-f flv "[Adresse de diffusion]]"
```

À l'heure actuelle, le module a été testé avec succès. Les principales limitations identifiées incluent la nécessité d'une connexion Internet stable avec un débit montant supérieur à 5 Mbps, l'absence de mécanisme de reconnexion automatique en cas de coupure, et une latence incompressible de 10 à 20 secondes due aux contraintes des plateformes de streaming.

Module d'interface utilisateur (GPIO)

Le module d'interface utilisateur assure l'interaction directe entre l'utilisateur et le système embarqué, sans dépendre d'un écran ou d'une interface logicielle externe. Il repose sur un ensemble de boutons physiques et de LEDs connectés aux GPIO de la Raspberry Pi, permettant de piloter les fonctions principales du dispositif et de fournir un retour visuel immédiat sur l'état du système.

Trois boutons physiques sont utilisés. Un bouton dédié permet de basculer dynamiquement entre le mode Wi-Fi Client et le mode Point d'Accès. Deux autres boutons contrôlent le streaming vidéo : l'un déclenche le démarrage du streaming et l'enregistrement vidéo, tandis que l'autre permet son arrêt immédiat et à éteindre. Ce découplage volontaire évite toute ambiguïté d'usage et limite les erreurs de manipulation lors de l'utilisation du système.

Le retour utilisateur est assuré par un ensemble de LEDs. Une LED RGB indique le mode réseau actif : la composante verte correspond au mode Client, tandis que la composante bleue signale le mode Point d'Accès. Deux LEDs simples complètent ce dispositif : une LED verte indique l'état du streaming (connexion en cours ou diffusion et enregistrement active), et une LED rouge est utilisée pour signaler les arrêts.

Le module est conçu pour fonctionner de manière autonome et asynchrone. Il supervise en permanence l'état du streaming et l'enregistrement, du réseau et des entrées utilisateur, tout en garantissant que certaines actions critiques (comme un changement de mode Wi-Fi) sont interdites lorsque le streaming est actif. Cette logique préventive permet d'éviter des états incohérents du système.

L'interface GPIO constitue ainsi une couche de contrôle locale robuste, offrant une expérience utilisateur simple, lisible et adaptée à un usage en conditions réelles, notamment lors de tournois ou d'événements sans accès direct à un clavier ou à un écran.

3.3. Architecture logicielle

L'architecture logicielle repose principalement sur trois scripts Bash :

- `cam_go_script.sh` : capture vidéo, création des fichiers sur la SD, création de noms horodatés,
- `net_mode.sh` : activation du mode AP ou du mode Client, gestion des services `hostapd`, `dnsmasq`, `dhcpcd` et `wpa_supplicant`,
- `livestream.sh` : infrastructure pour envoyer le flux vidéo en direct.

Une arborescence simple est adoptée pour garantir lisibilité et accès rapide :

```
/home/mao/scripts/  
  cam_go_script.sh  
  livestream.sh  
  net_mode.sh
```

```
/home/mao/videos/  
  YYYY-MM-DD_HH-MM-SS.h264
```

3.4. Architecture matérielle

Le matériel principal est organisé comme suit :

- une Raspberry Pi 3B+ jouant le rôle d'unité centrale ;
- une caméra ;
- une carte SD pour le stockage local ;
- des boutons permettant de démarrer ou arrêter un enregistrement ;
- des LEDs indiquant l'état du système ;
- une batterie externe alimentant le système en USB pour assurer la portabilité.

Le design final du boîtier intègre :

- un pied inclinable pour orienter la caméra ;
- une ouverture supérieure pour la lentille ;
- un espace intérieur pour la Raspberry et les câbles ;
- un accès facile à la carte SD.

4. Implémentation technique

Cette section détaille exhaustivement les aspects techniques de chaque module du système, en documentant précisément ce qui a été implémenté, testé et validé, ce qui reste instable malgré les efforts de stabilisation, et ce qui n'est pas encore opérationnel mais dont l'architecture est définie. L'approche adoptée suit une méthodologie d'ingénierie basée sur des cycles itératifs de développement-test-validation, avec une documentation systématique des décisions techniques, des résultats obtenus et des limitations identifiées.

4.1. Capture/stockage vidéo

La capture vidéo et la gestion du stockage local sont assurées par le script `cam_go_script.sh`. Ce script Bash constitue le cœur opérationnel du système d'enregistrement et a été conçu pour fonctionner de manière robuste lors de parties de Go de durée indéterminée.

Dès son lancement, le script initialise le répertoire de stockage `/home/mao/go_videos` et génère un horodatage unique correspondant au début de la session. Cet horodatage est utilisé à la fois pour nommer les segments temporaires et le fichier final, garantissant ainsi l'unicité des enregistrements et leur classement chronologique, sans dépendre d'un état externe.

L'enregistrement repose sur un pipeline temps réel combinant `rpicam-vid` et `ffmpeg`. L'outil `rpicam-vid` est configuré pour produire un flux vidéo en 1280×720 pixels à 25 images par seconde, encodé en H.264. L'option `-inline` force l'insertion régulière des en-têtes SPS/PPS, ce qui rend chaque segment indépendant et décodable sans information externe. Le flux est émis sur la sortie standard (`-o -`) afin d'être traité immédiatement par `ffmpeg`, sans écriture intermédiaire sur disque.

Le flux est ensuite pris en charge par `ffmpeg`, qui opère en copie directe (`-c:v copy`) afin d'éviter tout ré-encodage coûteux en ressources sur la Raspberry Pi 3B+. Le mode `segment` est utilisé pour découper l'enregistrement en fichiers de 10 secondes (`-segment_time 10`), avec une réinitialisation des horodatages (`-reset_timestamps 1`) afin d'assurer la cohérence temporelle de chaque fragment. Ce choix de segmentation fine permet de limiter les pertes de données en cas d'arrêt brutal ou de plantage du système.

La terminaison de l'enregistrement est gérée via un mécanisme explicite de capture des signaux système. Le script intercepte les signaux `SIGINT` et `SIGTERM` (notamment lors d'un appui sur `Ctrl+C`) à l'aide de la commande `trap`. Lorsqu'un arrêt est détecté, une fonction de finalisation est automatiquement appelée. Celle-ci concatène l'ensemble des segments correspondant à la session en un fichier final unique, puis supprime les fichiers temporaires afin de libérer l'espace disque. Ce mécanisme garantit que l'arrêt volontaire de l'enregistrement produit systématiquement un fichier exploitable.

Malgré son bon niveau de stabilité, ce script présente certaines limites. Aucune vérification n'est actuellement réalisée concernant l'espace disque disponible avant ou pendant l'enregistrement, ce qui peut entraîner une saturation de la carte SD. De plus, la concaténation repose sur une simple concaténation binaire (`cat`), adaptée au format H.264 brut mais non compatible avec d'autres conteneurs sans étape de traitement supplémentaire.

4.2. Réseau Client/AP

L'implémentation du mode point d'accès Wi-Fi ne repose pas uniquement sur des scripts de bascule, mais nécessite une préparation complète du système, incluant l'installation de services dédiés, la création de fichiers de configuration persistants et l'organisation explicite de l'arborescence logicielle. Cette section décrit l'ensemble des éléments concrets devant être présents sur la Raspberry Pi afin de garantir le fonctionnement du mode AP.

Création et organisation des fichiers de configuration

Le fonctionnement du point d'accès repose sur des fichiers de configuration système stockés dans l'arborescence standard de Linux. Le service `hostapd` utilise un fichier dédié, `/etc/hostapd/hostapd.conf`, décrivant les paramètres radio du point d'accès : interface réseau utilisée (`wlan0`), nom du réseau (SSID), canal Wi-Fi, type de chiffrement et mot de passe. Ce fichier est explicitement référencé dans `/etc/default/hostapd` afin d'indiquer au démon `systemd` quel fichier charger lors du démarrage du service.

Le service `dnsmasq` est configuré via le fichier `/etc/dnsmasq.conf`, dans lequel une plage d'adresses IP statiques est définie pour les clients connectés au réseau Wi-Fi autonome. La configuration limite volontairement cette plage afin d'éviter toute consommation inutile de ressources et de simplifier le diagnostic réseau. Le fichier de configuration par défaut est sauvegardé avant modification, permettant un retour rapide à l'état initial si nécessaire.

Ces fichiers de configuration sont conçus pour être persistants : ils ne sont pas recréés par les scripts et restent inchangés entre deux redémarrages du système.

Organisation des dossiers applicatifs

Afin de séparer clairement les données, les scripts et les journaux, une arborescence simple et explicite est adoptée. Les scripts exécutables sont regroupés dans le répertoire `/home/mao/scripts`, tandis que

les vidéos enregistrées sont stockées dans `/home/mao/go_videos`. Cette séparation permet d'éviter toute confusion entre code et données, et facilite la maintenance ou la migration du projet vers un autre support.

Activation et désactivation contrôlée des services

Les services `hostapd` et `dnsmasq` sont explicitement désactivés au démarrage du système. Ce choix est volontaire : le mode AP ne doit être activé que lorsque l'utilisateur le demande, afin d'éviter toute interférence avec le mode Client par défaut. L'activation effective de ces services est réalisée dynamiquement par les scripts de bascule réseau.

Lors du passage en mode AP, le script arrête les services de gestion Wi-Fi classiques, configure manuellement l'interface réseau avec une adresse IP statique, puis démarre les services nécessaires dans un ordre strict. À l'inverse, le retour en mode Client repose sur l'arrêt complet des services AP, le nettoyage de la configuration IP et la relance des services réseau standards.

Cette approche privilégie le contrôle explicite au détriment de l'automatisation complète, ce qui permet de mieux comprendre et diagnostiquer les dysfonctionnements observés.

Limites actuelles de l'implémentation

Bien que le système soit aujourd'hui pleinement fonctionnel dans ses scénarios d'usage principaux, certaines limites techniques ont été identifiées au cours des phases de test et d'intégration.

La gestion dynamique des modes réseau (Client ↔ Point d'Accès) reste sensible à l'ordre de démarrage des services système. En particulier, sur certains terminaux mobiles — notamment les appareils iOS — des comportements inattendus ont été observés lors de la connexion au point d'accès, pouvant provoquer une instabilité temporaire du réseau. Ces effets sont attribués à des interactions complexes entre les mécanismes de négociation réseau des terminaux clients et les services Wi-Fi embarqués de la Raspberry Pi.

4.3. Diffusion en direct

La diffusion en direct est implémentée dans le script `youtube - live.sh`. Celui-ci repose sur un pipeline similaire à celui de la capture locale, mais adapté aux contraintes des plateformes de streaming.

Le flux vidéo est capturé par `rpicas-vid` en 720p à 30 images par seconde, puis transmis à `ffmpeg`. Afin de satisfaire aux exigences de YouTube Live, un flux audio silencieux est généré à l'aide de la source virtuelle `anullsrc`. Le flux vidéo est encapsulé au format FLV après conversion du bitstream H.264 vers le format Annex-B, puis envoyé vers l'endpoint RTMP de la plateforme.

Les tests réalisés ont montré un fonctionnement correct du streaming, sous réserve d'une connexion Internet stable avec un débit montant suffisant. Aucune gestion automatique de reconnexion n'est actuellement implémentée, et toute coupure réseau entraîne l'arrêt immédiat de la diffusion.

4.4. Le serveur

Le module de diffusion via HTTP constitue l'interface principale de gestion du dispositif en mode autonome (*headless*). L'implémentation de ce module consiste à lancer un serveur **Python personnalisé héritant de la classe `http.server.SimpleHTTPRequestHandler`**.

En mode Access Point, le serveur est accessible via l'adresse IP statique `10.0.0.1` sur le port standard `80`. Outre la diffusion de contenu, il assure les fonctions d'administration suivantes :

- **Gestion des médias** : Indexation et téléchargement des fichiers vidéo (formats `.h264`, `.mp4`, `.mkv`) et des fichiers de parties SGF (Smart Game Format) générés par le système.
- **Configuration du Stream** : Interface permettant à l'utilisateur de définir l'URL RTMP de destination pour la diffusion en direct. Cette configuration est persistée dans un fichier texte pour être lue par le script de streaming.
- **Provisioning Wi-Fi** : Formulaire HTML permettant de saisir le SSID et le mot de passe d'un réseau local. Le backend génère dynamiquement les fichiers de connexion pour `NetworkManager` (avec

les permissions de sécurité 600) et déclenche un redémarrage système pour basculer du mode AP au mode Client.

- **Portail Captif** : Implémentation d'une logique de redirection (HTTP 302) interceptant toutes les requêtes inconnues pour rediriger automatiquement l'utilisateur vers la page de configuration principale.

État actuel : Opérationnel et stable.

4.5. Les boutons/Leds

L'implémentation de l'interface utilisateur repose sur un script Python autonome utilisant la bibliothèque RPi.GPIO. Ce script centralise la gestion des entrées physiques (boutons), des sorties visuelles (LEDs), ainsi que la coordination avec les scripts système de gestion du réseau et du streaming.

Les GPIO sont configurés en mode BCM afin d'assurer une correspondance claire entre le code et le brochage réel de la Raspberry Pi. Les LEDs sont déclarées en sortie numérique, tandis que les boutons sont configurés en entrée avec résistances de tirage internes (pull-up), garantissant un état stable en l'absence d'appui et limitant les parasites électriques.

Lors du démarrage du programme, une phase d'initialisation est exécutée afin de placer le système dans un état connu et sûr. Le mode Wi-Fi actif est détecté, puis les LEDs RGB sont mises à jour pour refléter l'état réseau courant. L'état interne du système est initialisé de manière explicite, en considérant qu'aucune capture ni diffusion n'est active au lancement. Une séquence visuelle de test est ensuite exécutée par l'allumage successif de l'ensemble des LEDs, permettant à l'utilisateur de vérifier immédiatement le bon fonctionnement du matériel sans consulter les journaux d'exécution.

La gestion des boutons repose sur un modèle événementiel. Chaque bouton est associé à une fonction de rappel déclenchée lors d'un appui, ce qui évite l'utilisation de boucles de scrutation continue et réduit la charge processeur. Le bouton vert est dédié au démarrage de la capture vidéo et, lorsque l'infrastructure réseau le permet, au lancement de la diffusion. Le bouton rouge implémente une double fonctionnalité selon la durée de l'appui. Un appui court provoque l'arrêt logiciel de la capture et de la diffusion, tandis qu'un appui prolongé déclenche une extinction sécurisée de la Raspberry Pi. Un délai logiciel est utilisé pour distinguer ces deux actions et prévenir les déclenchements accidentels.

Les LEDs jouent un rôle central dans le retour d'information à l'utilisateur et constituent un véritable langage visuel. La LED verte indique un fonctionnement nominal et n'est allumée que lorsque les processus attendus sont actifs. La LED rouge signale un état d'arrêt du système et est également utilisée pour indiquer des erreurs ou accompagner des phases critiques telles que l'arrêt. La LED RGB est dédiée à l'état du réseau et indique de manière explicite le mode Wi-Fi actif, vert pour le mode client et bleu pour le mode point d'accès. Les clignotements utilisés pour les transitions et les erreurs sont gérés de manière synchrone afin de conserver une logique simple et prévisible, sans recours à des mécanismes concurrents complexes. Dans son ensemble, cette interface matérielle fournit une interaction simple, robuste et explicite entre l'utilisateur et le dispositif embarqué. Le choix d'une architecture événementielle, d'un retour visuel minimaliste mais informatif et d'une gestion sécurisée des transitions permet d'assurer une exploitation fiable du système, y compris dans des conditions contraignantes telles qu'une perte de connectivité réseau ou une saturation de l'espace de stockage.

4.6. État de la partie

Un des défis de la conception de cet objet connecté était l'intégration d'un algorithme de computer vision pour analyser les différents flux vidéos enregistrés par la caméra de l'appareil. Pour ceci, nous avons repris le projet de 2024, que nous avons dû corriger et adapter à l'architecture de la Raspberry Pi. Pour assurer une meilleure reproductibilité de l'analyse, nous avons choisi de la conteneuriser avec Docker. Cela nous permet alors de compiler l'image pour l'architecture ARM, directement depuis nos ordinateurs personnels, pour gagner du temps dans le développement.

Cependant, nous avons rencontré plusieurs problèmes lors de l'implémentation de la computer vision.

1. La Raspberry Pi 3B+ n'est pas assez puissante pour faire tourner un lourd algorithme de reconnaissance d'images.
2. L'image Docker n'était pas fonctionnelle à cause de divers problèmes de librairies.

Par rapport au problème de puissance, une solution évidente aurait été de changer de matériel pour une carte ayant une plus grande puissance de calcul. Nous avons opté pour une deuxième solution : comme le groupe 20 livre un serveur connecté à notre objet, nous avons collaboré avec eux pour leur permettre de mettre point l'algorithme de computer vision sur le serveur. Cela permet de faire du *Cloud Computing* d'une certaine façon : nous déchargeons le poids de l'analyse de vidéos au serveur, plus puissant et mieux refroidi.

Cette solution comporte un problème, dans le cas où l'on souhaite utiliser l'objet en local, sans se connecter au serveur, nous ne pouvons pas analyser la vidéo. C'est pourquoi nous avons malgré tout implémenté l'image Docker pour l'analyse de vidéos, même si elle serait particulièrement lente, sur la Raspberry Pi, et l'analyse se lance donc à la fin d'un enregistrement local de vidéo, automatiquement. Une autre solution consisterait en le téléchargement de la vidéo localement, suivi d'une upload au serveur, plus tard, quand l'utilisateur disposera d'un accès internet, afin de faire l'analyse sur le serveur à posteriori.

5. Résultats

Cette section présente l'état réel de fonctionnement du système à l'issue de la phase d'implémentation et de tests. Les résultats sont évalués de manière pragmatique, en distinguant les fonctionnalités pleinement validées, celles partiellement opérationnelles et celles qui n'ont pas encore atteint un niveau de fonctionnement satisfaisant.

La caméra embarquée a été entièrement validée. La capture vidéo fonctionne de manière fiable avec le module caméra de la Raspberry Pi, et le flux généré est conforme aux paramètres définis (résolution, fréquence d'images et encodage H.264). Aucun dysfonctionnement majeur n'a été observé lors des tests prolongés, sous réserve d'une connexion matérielle correcte du module caméra.

Le mécanisme d'enregistrement local est également validé. Les vidéos sont correctement segmentées pendant l'acquisition, puis concaténées automatiquement lors de l'arrêt de l'enregistrement. Les fichiers finaux générés sont exploitables et correctement horodatés, garantissant une organisation claire des données. Les tests confirment que l'arrêt volontaire de l'enregistrement produit systématiquement un fichier vidéo final cohérent.

Le mode Wi-Fi Client est pleinement opérationnel. La Raspberry Pi se connecte de manière fiable à un réseau Wi-Fi existant et bénéficie d'un accès Internet stable, condition nécessaire au fonctionnement du streaming en direct. Ce mode a été testé avec succès dans plusieurs environnements réseau et constitue le mode de fonctionnement par défaut du système.

Le mode Point d'Accès (AP) constitue le principal résultat fonctionnel en termes d'interaction utilisateur. Lorsqu'il est activé, le système génère son propre réseau Wi-Fi, permettant à un terminal externe de s'y connecter directement. Ce mode assure un accès fiable au site web embarqué, à partir duquel l'utilisateur peut consulter l'interface, récupérer les vidéos enregistrées localement et configurer les paramètres réseau du dispositif.

Le site web embarqué permet notamment de renseigner les identifiants d'un réseau Wi-Fi externe (nom du réseau et mot de passe). Cette fonctionnalité permet de préparer l'intégration ultérieure du système à un réseau existant, en facilitant le basculement vers un mode client sans intervention directe sur la Raspberry Pi. La récupération des vidéos est pleinement fonctionnelle via cette interface, ce qui valide la chaîne de capture, de stockage et d'accès aux données.

Concernant la diffusion en direct, le streaming vidéo est opérationnel au niveau système, mais son intégration dans l'interface web reste partielle. Le lien vers le flux de diffusion n'est pas encore automatiquement intégré au site web et doit être fourni manuellement pour être consulté. Cette limitation concerne uniquement l'exposition du flux à l'utilisateur final et non le fonctionnement du mécanisme de streaming lui-même.

Dans l'ensemble, les résultats obtenus démontrent la faisabilité technique du projet et valident les choix d'architecture principaux. Les fonctionnalités critiques liées à la capture et à l'enregistrement vidéo sont pleinement opérationnelles, tandis que les aspects réseau avancés restent partiellement aboutis et constituent des perspectives de travail futures.

6. Perspectives et Amélioration

Les perspectives d'amélioration du système TenukiGo concernent à la fois les aspects matériels, logiciels et l'expérience utilisateur. La première limitation identifiée porte sur la gestion de la mémoire de stockage. Les captures vidéo sont actuellement segmentées en séquences de dix secondes, chaque segment représentant environ dix mégaoctets. Sur une carte micro-SD de capacité limitée, cette consommation entraîne rapidement une saturation de l'espace disque, pouvant provoquer un arrêt non contrôlé de la capture. Une amélioration consisterait soit à optimiser les paramètres de capture (résolution, débit, fréquence d'images), soit à déporter le stockage vers un support externe, soit à mettre en place une surveillance automatique de l'espace disque afin d'anticiper un arrêt propre avant saturation.

Sur le plan matériel, le déploiement du système sur une Raspberry Pi plus performante, telle qu'un modèle Raspberry Pi 4, constitue une piste d'amélioration pertinente. Une plateforme disposant de davantage de mémoire vive et de meilleures performances de calcul permettrait de renforcer la stabilité globale du système, de faciliter l'exécution simultanée des différentes briques logicielles (capture, streaming, analyse) et d'ouvrir la voie à une intégration plus poussée des traitements directement sur la carte.

Concernant la connectivité réseau, une amélioration importante concerne le mode Point d'Accès (AP), en particulier pour assurer une compatibilité plus robuste avec les terminaux iOS. Bien que le mode AP permette actuellement l'accès au site web embarqué et la récupération des vidéos, des ajustements sont nécessaires afin de garantir un fonctionnement homogène sur l'ensemble des navigateurs mobiles, notamment en ce qui concerne la gestion des flux et des redirections réseau.

Enfin, une évolution majeure du système consisterait à intégrer directement sur la Raspberry Pi l'ensemble de la chaîne de diffusion et de traitement des données. Cela inclut l'automatisation complète de l'exposition du lien de streaming au sein de l'interface web, ainsi que la génération et la mise à disposition du fichier SGF directement depuis le dispositif embarqué, sans dépendance à des étapes manuelles ou à des services externes. Une telle intégration renforcerait l'autonomie du système et améliorerait significativement l'expérience utilisateur finale.

Dans leur ensemble, ces perspectives s'inscrivent dans une continuité logique du travail réalisé et visent à transformer un prototype fonctionnel en un système plus robuste, autonome et adapté à une utilisation prolongée dans des conditions réelles.

OUR WORLDWIDE PARTNERS UNIVERSITIES - DOUBLE DEGREE AGREEMENTS



3 CAMPUS



IMT Atlantique Bretagne-Pays de la Loire – <http://www.imt-atlantique.fr/>

Campus de Brest

Technopôle Brest-Iroise
CS 83818
29238 Brest Cedex 3
France
T +33 (0)2 29 00 11 11
F +33 (0)2 29 00 10 00

Campus de Nantes

4, rue Alfred Kastler
CS 20722
44307 Nantes Cedex 3
France
T +33 (0)2 51 85 81 00
F +33 (0)2 99 12 70 08

Campus de Rennes

2, rue de la Châtaigneraie
CS 17607
35576 Cesson Sévigné Cedex
France
T +33 (0)2 99 12 70 00
F +33 (0)2 51 85 81 99



IMT Atlantique

Bretagne-Pays de la Loire
École Mines-Télécom